

ARQUITECTURA DE MICROSERVICIOS

PATRÓN SAGA

Para la Creación de Procesos de Selección de Personal CAS

Orquestación distribuida con RabbitMQ · MySQL · MongoDB · Spring Boot

ms-proceso-orquestador

ms-proceso-service

ms-proceso-comite-service

ms-notification-service

Autor: PERCY GIANCARLO GUILLEN ALCANTARA

¿Qué es el Patrón SAGA?

DEFINICIÓN

El Patrón SAGA es una estrategia para gestionar transacciones distribuidas en arquitecturas de microservicios, donde cada paso local de la transacción publica un evento o mensaje que desencadena el siguiente paso.

COMPENSACIÓN

Si un paso falla, se ejecutan transacciones compensatorias para deshacer los cambios anteriores y mantener la consistencia del sistema.

TIPOS DE SAGA

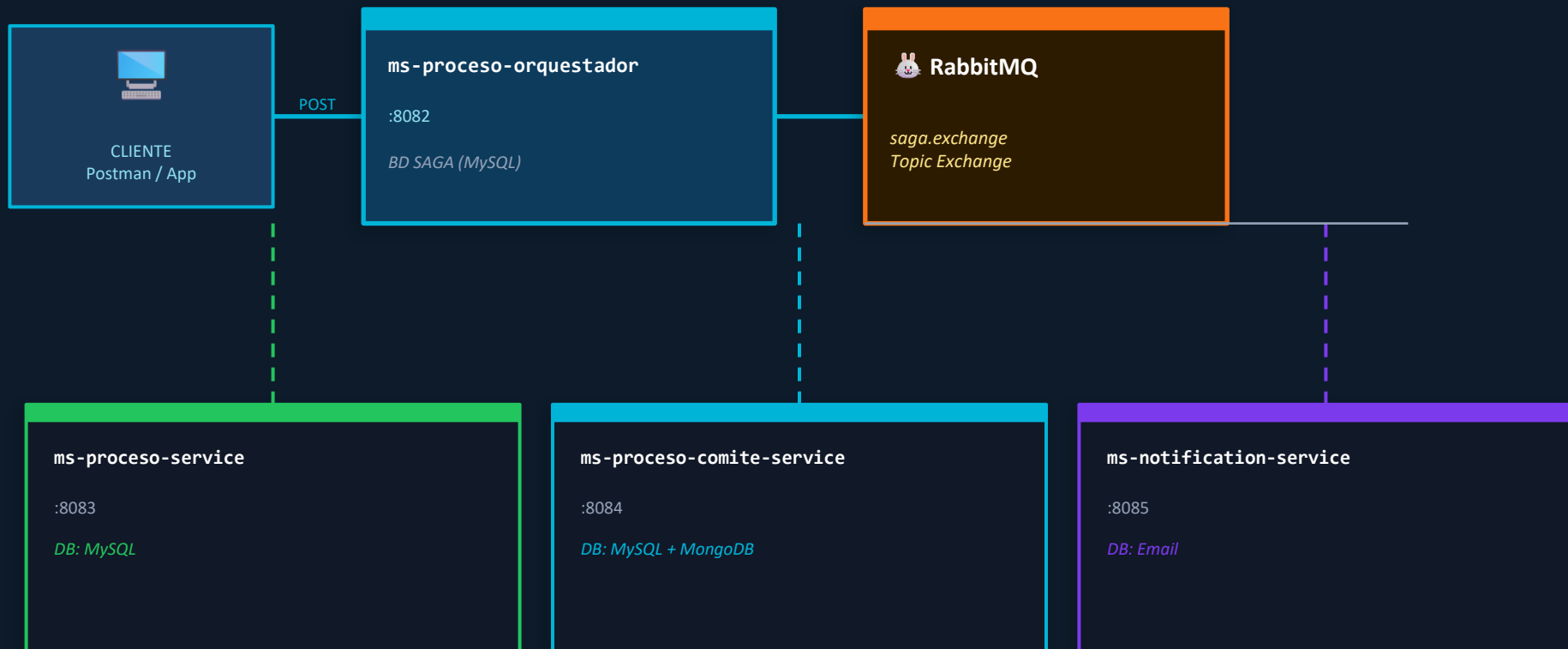
COREOGRAFÍA

Los microservicios se comunican directamente entre sí mediante eventos. Sin orquestador central.

ORQUESTACIÓN ← Este caso

Un orquestador central controla el flujo de la SAGA, enviando comandos a cada microservicio y manejando fallos centralmente.

Arquitectura General del Sistema



PASO 1 · Cliente → Orquestador

1

POST http://localhost:8082/api/saga/proceso

```
{
  "nombre": "PROCESO CAS N° 999-2026-PCM-OGRH",
  "resumen": "CONVOCATORIA PARA LA CONTRATACIÓN...",
  "titulo": "2DA. CONVOCATORIA",
  "cantidad": 21,
  "fechaInicio": "2026-03-04",
  "fechaFin": "2026-03-04",
  "organoId": "79e69473-...",
  "listaProcesoComite": [
    { "organoId": "79e...", "tipo": 3.00,
      "nombre": "PERCY GUILLEN", "email": "..." },
    { "organoId": "62...", "tipo": 1.00,
      "nombre": "JUAN PEREZ", "email": "..." }
  ]
}
```

RESPONSE 200 OK

```
{
  "sagaId": "411d094d-...",
  "message": "Saga started successfully"
}
```

El Orquestador realiza:

- 1 Genera un sagaId único (UUID)
- 2 Guarda payload del comité en BD SAGA
- 3 Estado inicial: STARTED
- 4 Publica → proceso.create.queue en RabbitMQ



Lee cola: `proceso.create.queue`



Orquestador lee: `proceso.created.queue`



FLUJO EXITOSO

- Recibe CreateProcesoCommand de la cola
- Registra el proceso en MySQL (tb_proceso)
- Genera evento ProcesoCreatedEvent
- Publica → proceso.created (routing key)
- Cola destino: proceso.created.queue



FLUJO DE ERROR

- Ocorre una excepción al registrar
- Captura el error con try/catch
- Genera ProcesoCreationFailedEvent
- Publica → proceso.creation.failed
- Cola: proceso.creation.failed.queue

Orquestador: actualiza estado → PROCESO_CREATED · recupera payload del comité desde BD SAGA · publica proceso.comite.create.queue

Acciones del servicio

Lee cola:	proceso.comite.create.queue
Recibe:	CreateProcesoComiteCommand con procesoId
Guarda comité:	MySQL → tb_proceso_comite
Guarda en MongoDB:	ProcesoComiteDocument
Publica evento:	proceso.comite.created
En error:	proceso.comite.creation.failed

MongoDB — ProcesoComiteDocument

```
{
  "_id": ObjectId("..."),
  "idProcesoComite": "uuid",
  "organo": {
    "id": "uuid",
    "codigo": "PCM-OGRH",
    "nombre": "Oficina de RRHH"
  },
  "tipo": 3.00,
  "nombre": "PERCY GUILLEN",
  "email": "cyper2@hotmail.com",
  "fechaCreacion": "2026-03-14T..."
}
```

Orquestador: actualiza estado → COMITE_CREATED · publica proceso.comite.notification.queue con datos del comité

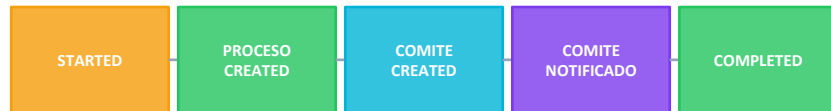
Flujo de Notificación

- 1 Lee proceso.comite.notification.queue
- 2 Obtiene lista de integrantes del comité
- 3 Genera email personalizado por integrante
- 4 Envía correo a cada miembro del comité
- 5 Registra notificación enviada
- 6 Publica: proceso.comite.notificated
- 7 En error: proceso.comite.notification.failed

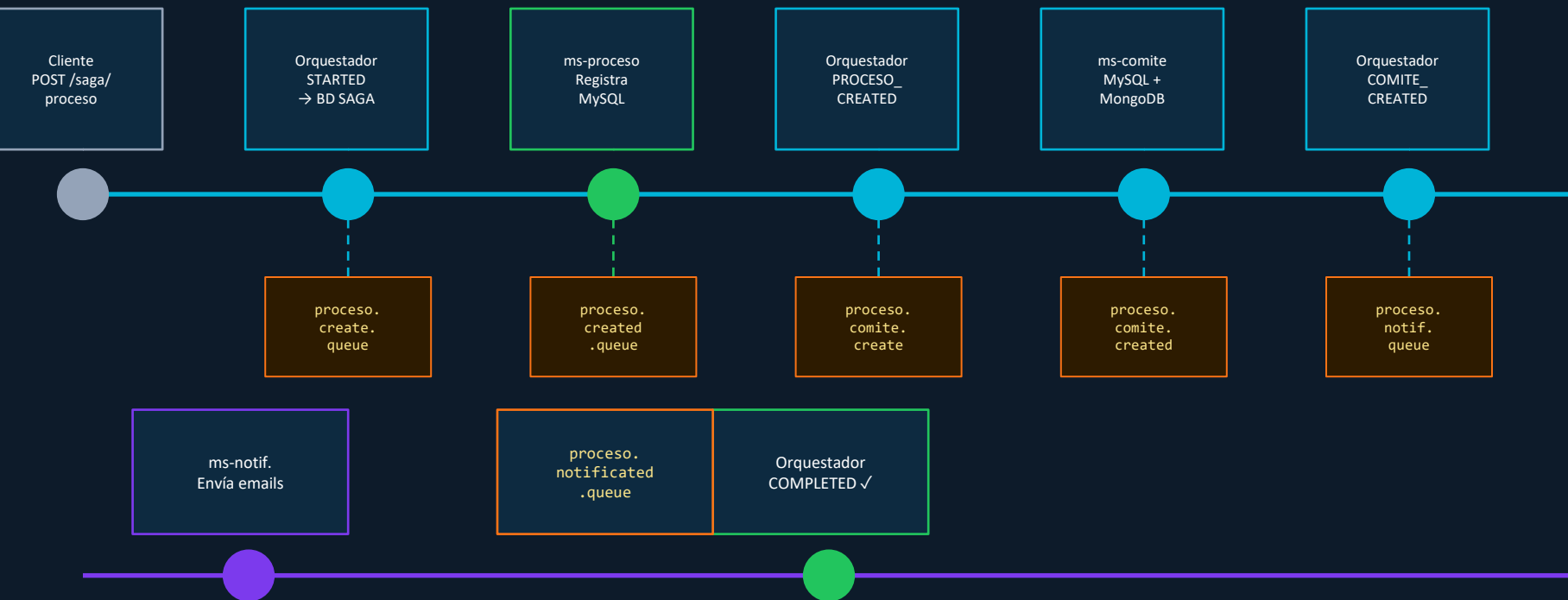
✓ SAGA COMPLETADA

El orquestador recibe proceso.comite.notificated
Actualiza estado → COMITE_NOTIFICADO
Actualiza estado final → COMPLETED
Persiste en BD SAGA — Caso de uso finalizado

Estados de la SAGA



Flujo Completo del Patrón SAGA



⚠️ **COMPENSACIÓN:** Si falla ms-comite → Orquestador publica proceso.delete.queue → ms-proceso elimina el registro → Estado: COMPENSATED

Colas RabbitMQ · Routing Keys · Topic Exchange

Routing Key	Cola (Queue)	Consumidor	Tipo
proceso.create	proceso.create.queue	ms-proceso-service	COMANDO
proceso.created	proceso.created.queue	ms-proceso-orquestador	EVENTO
proceso.creation.failed	proceso.creation.failed.queue	ms-proceso-orquestador	EVENTO ERR
proceso.comite.create	proceso.comite.create.queue	ms-proceso-comite-service	COMANDO
proceso.comite.created	proceso.comite.created.queue	ms-proceso-orquestador	EVENTO
proceso.comite.creation.failed	proceso.comite.creation.failed.queue	ms-proceso-orquestador	EVENTO ERR
proceso.comite.notification	proceso.comite.notification.queue	ms-notification-service	COMANDO
proceso.comite.notificated	proceso.comite.notificated.queue	ms-proceso-orquestador	EVENTO
proceso.comite.notification.failed	proceso.comite.notification.failed.queue	ms-proceso-orquestador	EVENTO ERR
proceso.delete	proceso.delete.queue	ms-proceso-service	COMPENSACIÓN
proceso.deleted	proceso.deleted.queue	ms-proceso-orquestador	COMPENSACIÓN

Beneficios y Conclusiones

Consistencia Eventual

Cada microservicio mantiene su propia BD y la consistencia se logra de forma eventual mediante eventos y compensaciones.

Alta Disponibilidad

Si un microservicio falla, el orquestador detecta el fallo y ejecuta la compensación sin perder la trazabilidad del proceso.

Trazabilidad Total

BD SAGA registra cada estado de la transacción: STARTED → PROCESO_CREATED → COMITE_CREATED → COMPLETED.

Compensación Automática

Ante fallos en ms-comite, el orquestador revierte automáticamente el proceso creado mediante la cola proceso.delete.queue.

Desacoplamiento Total

RabbitMQ garantiza que los microservicios nunca se comunican directamente, reduciendo el acoplamiento al mínimo.

Persistencia Dual

ms-proceso-comite-service persiste en MySQL (operacional) y MongoDB (lectura rápida), optimizando cada caso de uso.